Übungsblatt 7: Performance und Pipelining

Mikrocomputertechnik 1

Musterlösung

Zielsetzung und Arbeitsweise

Sie bestimmen Laufzeiten, zeichnen Stufenfolgen und begründen, welcher Datenwert wann verfügbar ist. Teil A ist ohne Simulator lösbar und übt Klausuraufgaben. Teil B vergleicht vorhergesagte Abläufe mit Ripes.

Orientierung: Teil A etwa 70 Minuten und 60 Punkte zur Selbsteinschätzung, Teil B etwa 25 Minuten. Für eine kürzere gemeinsame Arbeitsphase eignen sich Aufgaben 1, 2 und 4; die übrigen Aufgaben können als Vertiefung bearbeitet werden. Geben Sie bei Rechnungen Größen, Einheiten und Zählgrenzen an.

Vereinbartes Modell

- RV32I, unkomprimierte Vier-Byte-Instruktionen, gültige wortausgerichtete Datenadressen. Anfangswerte sind vor dem betrachteten Kern hergestellt.
- Fünf Stufen: IF (Instruction Fetch, Befehlsholen), ID (Instruction Decode, Decodieren und Registerlesen), EX (Execute, Rechnen), MEM (Memory Access, Datenzugriff), WB (Write Back, Rückschreiben / Abschluss).
- Anfangs leere Pipeline, höchstens ein neuer Befehl je Takt, getrennte Befehls- und Datenzugriffe. Ein Datenzugriff braucht eine MEM-Stufe.
- Forwarding aus EX/MEM und MEM/WB für bereite Werte, getrennt für benutzte Quellen. Jüngster passender gültiger Registerschreiber gewinnt; Ziel x0 löst keine Weiterleitung aus.
- Registerbank: Ein am Taktende geschriebenes Register ist erst im Folgetakt lesbar. Ohne zusätzlichen Lesebypass muss die Erkennung einen gleichzeitigen WB/ID-Konflikt halten.
- Load-Use bei unmittelbarer EX-Nutzung: PC und IF/ID halten, Bubble nach ID/EX, ältere Instruktionen weiterlaufen lassen.
- Branch-Entscheidung in EX, zunächst sequenzieller Fetch. Genommener Branch: zwei jüngere Einträge verwerfen. Vergleichswerte sind in den Branch-Aufgaben bereits bereit.
- Zählung: Beginn des ersten IF bis Ende des letzten WB. Nur gültig abgeschlossene Maschineninstruktionen gehören zu N; Bubble und verworfene Arbeit nicht.

Für Aufgabe 1 und 4: reine Stufenzeiten 200, 100, 150, 250 und 100 ps, effektiver Registeroverhead 20 ps je Taktpfad. Das sind didaktische Annahmen.

Teil A: Papieraufgaben

Aufgabe 1: Laufzeit, Latenz und Durchsatz (12 Punkte)

1. Bestimmen Sie aus den angegebenen Stufenzeiten die Taktperiode des Single-Cycle-Modells und der Pipeline. Begründen Sie Summe und Maximum. (3 P.)
2. Bestimmen Sie die Latenz eines einzelnen Befehls in beiden Modellen. Warum kann eine höhere Einzellatenz trotzdem nützlich sein? (2 P.)
3. Zwanzig unabhängige Maschineninstruktionen sollen abschließen. Bestimmen Sie Taktzahl, endlichen CPI und Gesamtzeit beider Modelle sowie den Pipeline-Speedup. (5 P.)
4. Nennen Sie zwei Voraussetzungen für den idealen Grenzwert von Faktor fünf. Ist er im vorgegebenen Zeitenmodell erreichbar? (2 P.)

Musterlösung Aufgabe 1

1. $T_{\text{single}} = 200 + 100 + 150 + 250 + 100 + 20 = 820$ ps. Der längste angenommene Signalpfad läuft durch alle Bereiche. $T_{\text{pipe}} = \max(200, 100, 150, 250, 100) + 20 = 270$ ps. In der Pipeline begrenzt die längste Stufe plus Overhead die Periode.
2. $L_{\text{single}} = 820$ ps, $L_{\text{pipe}} = 5 \cdot 270 = 1350$ ps. Eine längere Latenz eines einzelnen Befehls kann mit höherem Durchsatz vieler überlappter Befehle einhergehen.
3. Single Cycle: $C = 20$, CPI $= 1$, $t = 16400$ ps. Pipeline: $C = 20 + 4 = 24$, CPI $= 24/20 = 1{,}2$, $t = 6480$ ps. $S = 16400/6480 \approx 2{,}53$. Füllung und Leerung werden gemeinsam nur einmal als vier Zusatztakte berücksichtigt.
4. Beispielsweise gleich lange Stufen und vernachlässigbarer Registeroverhead; außerdem lange Folge ohne Zusatzzyklen. Hier ist der Grenzwert $820/270 \approx 3{,}04$, deshalb auch bei sehr großem N kein Faktor fünf.

Aufgabe 2: Load-Use-Raster und Werte (12 Punkte)

Anfang: `a0` zeigt auf ein gültiges Wort mit Wert 42. Die übrigen genannten Register beginnen bei null. I1, I2 und I3 bezeichnen die drei Zeilen; sie sind keine zusätzlichen Instruktionen.

```
lw   t0, 0(a0)      # I1
addi t1, t0, 1      # I2
add  t2, t1, t1     # I3
```

1. Füllen Sie das Raster aus. Ein gehaltenes ID- oder IF-Feld muss als solches erkennbar sein; verwenden Sie dafür `ID hält` beziehungsweise `IF hält`. (5 P.)

2. Nennen Sie für I2 in EX und beide Quellen von I3 in EX jeweils den Datenwert und das liefernde Pipeline-Register. Warum darf I2 nicht die Load-Adresse erhalten? (3 P.)
3. Bestimmen Sie die Endwerte t0, t1 und t2. (2 P.)
4. Bestimmen Sie N, C und den endlichen CPI. Zählt die Bubble zu N? (2 P.)

Musterlösung Aufgabe 2

Eintrag / Takt	1	2	3	4	5	6	7	8
I1	IF	ID	EX	MEM	WB			
I2		IF	ID	ID hält	EX	MEM	WB	
I3			IF	IF hält	ID	EX	MEM	WB
Bubble				EX	MEM	WB		

2. I2 erhält in EX5 den geladenen Wert 42 aus MEM/WB. I3 erhält in EX6 beide Registeroperanden mit Wert 43 aus EX/MEM. EX des Loads bildet nur die Adresse; diese ist kein Ersatz für das Wort 42.
3. t0 ist 42, t1 ist $42 + 1 = 43$, t2 ist $43 + 43 = 86$.
4. $N = 3$, $C = 8$, CPI $= 8/3 \approx 2{,}67$. Die Bubble ist kein zusätzlich abgeschlossener Maschinenbefehl. Es gibt einen Load-Use-Zusatzzyklus, danach keinen weiteren Konflikt der benutzten Quellen.

Aufgabe 3: Forwarding begründen (8 Punkte)

Drei voneinander unabhängige Fälle:

A: Zwei passende Produzenten

Anfang: `t1 = 7, t2 = 3, t4 = 4.`

```
add  t0, t1, t2
addi t0, t0, 1
sub  t3, t0, t4
```

Wenn `sub` in EX ist, steht `addi` in MEM und `add` in WB. Welche Quelle muss den ersten Operanden liefern und welcher Endwert entsteht in t3? Begründen Sie den Vorrang. (3 P.)

B: Ziel x0

Ersetzen Sie im folgenden eigenständigen Fall die mittlere Zeile durch `add x0, t0, t0`:

```
add t0, t1, t2
add x0, t0, t0
sub t3, x0, t4
```

Warum darf der interne Rechenwert 20 nicht nach `sub` weitergeleitet werden? Welcher Wert entsteht? (3 P.)

C: Gleichzeitig WB und ID

Ein Schreiber ist in WB, sein Leser nach zwei unabhängigen Instruktionen in ID. Genügen die beiden EX-Forwarding-Pfade allein unter unserer Registerbankkonvention? Nennen Sie die nötige Behandlung und eine alternative Modellannahme. (2 P.)

Musterlösung Aufgabe 3

A: add erzeugt 10, addi erzeugt 11. Der jüngere Schreiber addi in EX/MEM gewinnt vor dem älteren add in MEM/WB. sub nutzt 11 und schreibt $11 - 4 = 7$ nach t3.

B: x0 bleibt null und verwirft den Registerschreibversuch. Das interne Ergebnis 20 ist deshalb kein sichtbares Ergebnis für einen Leser von x0. Die Prüfung rd ungleich x0 unterdrückt die Weiterleitung. sub schreibt $0 - 4 = -4$, das Bitmuster 0xFFFFFFFC.

C: Nein. Im Folgetakt erreicht der Leser EX, aber der Schreiber steht nicht mehr in MEM/WB. Ohne zusätzlichen Lesebypass muss der Leser einen Takt in ID gehalten werden und danach neu lesen. Eine Registerbank mit Schreiben vor Lesen im selben Takt oder ein zusätzlicher WB-nach-ID-Bypass würde den Fall anders behandeln.

Aufgabe 4: Flush, Endzustand und Gesamtkosten (10 Punkte)

Anfang: `t0 = t1 = 1, t2 = 77, t3 = 0, a0 = 0`. s1 zeigt auf ein beschreibbares Wort mit Wert null.

```
beq  t0, t1, Ziel
sw   t2, 0(s1)
addi t3, x0, 99
Ziel:
addi a0, x0, 7
```

1. Zeichnen Sie das Stufenraster vom IF des Branch bis zum WB der Zielinstruktion. Kennzeichnen Sie, wann die beiden Folgeinstruktionen ungültig werden. (4 P.)
2. Bestimmen Sie Speicherwort, t3 und a0 nach dem Abschluss der Zielinstruktion. Warum genügt es nicht, nur den Fetch umzulenken? (2 P.)
3. Ein anderer, längerer Lauf schließt $N = 100$ Instruktionen ab. Er hat zehn zusätzliche Wartezyklen und zwanzig Branches, davon zwölf genommen. Vergleichswerte sind bereit, Zusatzkosten überdecken sich nicht. Bestimmen Sie C, CPI und Laufzeit bei 270 ps. (4 P.)

Musterlösung Aufgabe 4

Eintrag / Takt	1	2	3	4	5	6	7	8
beq	IF	ID	EX	MEM	WB			
sw		IF	ID	ungültig				
addi t3			IF	ungültig				
addi a0				IF	ID	EX	MEM	WB

An der Grenze zu Takt 4 werden IF/ID und ID/EX ungültig; die Folgeeinträge dürfen danach keine aktive Zustandswirkung haben.

2. Speicherwort 0, t3 gleich 0 und a0 gleich 7. Der falsche Store und die falsche Registeraddition würden trotz neuem Fetch später schreiben, wenn ihre schon gespeicherten Einträge gültig blieben.
3. Bei Not-Taken-Annahme sind $F = 12$ Pfade falsch. $C = N + 4 + H + 2F = 100 + 4 + 10 + 24 = 138$. CPI $= 138/100 = 1{,}38$. $t = 138 \cdot 270 = 37260\,\text{ps} = 37{,}26\,\text{ns}$. Branches sind bereits in N enthalten; verworfene Instruktionen erhöhen N nicht.

Aufgabe 5: Scheduling mit derselben Arbeit (10 Punkte)

Das Wort an der gültigen Adresse in s1 ist 42. Anfang: a5 gleich null.

```
lw   t0, 0(s1)
add  a0, t0, t0
addi a5, a5, 1
```

1. Ordnen Sie genau diese drei Instruktionen so um, dass die unmittelbare Load-Nutzung entfällt. Begründen Sie, welche Register- und Speicherabhängigkeiten die Umordnung erlauben. (3 P.)
2. Bestimmen Sie die Endwerte in t0, a0 und a5 sowie die Taktzahl vor und nach der Umordnung. Wie hoch ist der Kern-Speedup bei gleicher Periode? (3 P.)
3. Würde das zusätzliche Einfügen von `addi a5, a5, 1` zwischen Load und add denselben Vergleich liefern? Begründen Sie. (2 P.)
4. In einer anderen Variante lautet die letzte Zeile `addi s1, s1, 4`. Warum darf diese Erhöhung nicht vor den Load rücken? Das nächste Wort kann einen anderen Wert haben. (2 P.)

Musterlösung Aufgabe 5

1. Zulässig ist lw, dann addi a5, dann add. a5 ist unabhängig vom Basisregister s1, dem Load-Ergebnis t0 und dem Ergebnis a0. Die Erhöhung greift nicht auf den Speicher zu; die Reihenfolge Load vor Nutzung bleibt erhalten.
2. In beiden Folgen: t0 gleich 42, a0 gleich 84, a5 gleich 1. Vorher $C = 3 + 4 + 1 = 8$, danach $C = 3 + 4 = 7$. $S = 8/7 \approx 1{,}14$ für dieses Kernfenster.
3. Nein. Beim Einfügen zusätzlich zur schon vorhandenen Erhöhung gäbe es vier statt drei Instruktionen und a5 würde 2 statt 1. Die Aufgabe und das Ergebnis wären verändert.
4. Die vorgezogene Erhöhung würde die Adresse des Loads um vier Byte verschieben. Er würde ein anderes Wort lesen. Beispielsweise 99 statt 42 ergäbe 198 statt 84 in a0. Ein Verschieben nur zwischen Load und add ist unter den angegebenen Annahmen möglich; vor den Load ist es nicht zulässig.

Aufgabe 6: Sprungvorhersage als Zustandsaufgabe (8 Punkte)

Ein isolierter Branch mit der tatsächlichen Folge T, T, T, N. T bedeutet genommen, N nicht genommen. Die Richtungstabelle teilt ihren Eintrag nicht mit anderen Branches; das Ziel ist für diesen Vergleich rechtzeitig verfügbar. Die Korrekturzeit pro Fehler wird mit zwei Takten angenommen.

1. Ein Einbitmodell startet mit Vorhersage N. Nach jeder Entscheidung speichert es die tatsächliche Richtung als nächste Vorhersage. Füllen Sie die Tabelle und bestimmen Sie Fehlerzahl und Zusatzzyklen. (4 P.)

Ausführung	1	2	3	4
Tatsächlich	T	T	T	N
Vorhergesagt				
Fehler?				

2. Ein Zweibitzähler startet in 11. Bei T erhöht er bis höchstens 11, bei N vermindert er bis mindestens 00. 00 und 01 sagen N vorher, 10 und 11 sagen T vorher. Bestimmen Sie Vorhersagen, Endzustand und Fehlerzahl für dieselbe Folge. (3 P.)
3. Ist der Zweibitzähler aufgrund dieses Ergebnisses für jedes Programm überlegen? Begründen Sie. (1 P.)

Musterlösung Aufgabe 6

1. Vorhersagen N, T, T, T. Fehler bei Ausführung 1 und 4: zwei Fehler, vier zusätzliche Takte. Endzustand N.
2. Vorhersagen T, T, T, T. Die ersten drei T halten den Zustand jeweils in 11; das letzte N führt nach 10. Es gibt einen Fehler und zwei Zusatzzyklen.
3. Nein. Die beiden Ausgangszustände, die kurze tatsächliche Folge und die angenommene Korrekturzeit bestimmen diesen Vergleich. Andere Verläufe oder Anfangszustände können andere Fehlerzahlen liefern. Richtung allein stellt zudem noch keine Zieladresse bereit.

Teil B: Ripes

Aufgabe 7: Vorhersage und Beobachtung vergleichen

Wählen Sie eine fünfstufige RV32-Variante mit Forwarding und Hazard-Erkennung, komprimierte Instruktionen aus. Dokumentieren Sie Version und Modell. Setzen Sie vor jedem Versuch zurück. Verwenden Sie einzelne Clock-Schritte und die Stufenhistorie Ihrer Version; Menünamen und Farben können abweichen.

Öffnen Sie die vollständigen Programme im Ordner `beispiele/`. Initialisierung, `la`-Expansion, echte Programm-NOPs und Abschluss gehören nicht zum relativen Kernfenster. Beginnen Sie die Kernzählung beim IF der mit `Kern` markierten Folge. Eine Prozessorkonfiguration kann vom Papiermodell abweichen; begründen Sie dann die beobachtete Stufenzahl statt eine passende Anzeige zu erzwingen.

1. `vl7_forwarding.s`: Identifizieren Sie add und sub im Kern. Wenn sub in EX ist, notieren Sie seine Datenquelle und den Operand mit Wert 150. Prüfen Sie nach dem passenden WB die Endwerte t2 und a0.
2. `vl7_loaduse.s`: Identifizieren Sie Load und Verdopplung. Notieren Sie den gehaltenen Verbraucher, den weiterlaufenden Load und die wirkungslose Lücke. Prüfen Sie die beiden benutzten Additionsquellen sowie t0 und a0 nach Abschluss.
3. `vl7_flush.s`: Bestimmen Sie Vergleichs- und Zielstufe Ihrer Variante. Prüfen Sie, dass falscher Store und falsche Addition keine bestätigte Wirkung haben. Lesen Sie Speicherwort, t3 und a0 nach der Zielinstruktion.
4. `vl7_schedule_a.s` und `vl7_schedule_b.s`: Vergleichen Sie dieselben drei Kerninstruktionen. Prüfen Sie t0, a0 und a5. Dokumentieren Sie, welche Zusatzzyklen entfallen; verwenden Sie den globalen Laufzeitzähler nicht ungeprüft als Kernzählung.

Ein Selektorwert oder eine Kabelfarbe allein ist kein Nachweis. Geben Sie Produzent, lieferndes Register, Datenwert und Verbraucher an. Der Codeabschluss verwendet einen Ripes-Umgebungsdienst; prüfen Sie dessen Vertrag in der Dokumentation Ihrer Version. Die Aufgabe kann vor dem Abschlussdienst beendet werden, sobald die Kern-Ergebnisse vorliegen.

Musterlösung Aufgabe 7: Erwartungen, keine protokollierten Ripes-Läufe

Forwarding: t2 wird 150, a0 wird 100. Das unmittelbare Rechenpaar erhält den neuen ersten sub-Operanden aus EX/MEM ohne Zusatzzyklus im Lehrmodell.

Load Use: t0 wird 42, a0 wird 84. Beide Quellen der Verdopplung brauchen 42. Im Lehrmodell wird sie einen Takt in ID gehalten und erhält danach das Wort aus MEM/WB; die Load-Adresse ist kein Nutzdatenersatz.

Flush: Das Wort bei slot bleibt 0, t3 bleibt 0 und a0 wird 7. Falsche jüngere Einträge müssen vor Zustandswirkungen ungültig werden. Auflösungsstufe und Darstellung sind anhand der konkreten Simulatorvariante zu dokumentieren.

Scheduling: Beide Varianten enden mit t0 gleich 42, a0 gleich 84 und a5 gleich 1. Für das anfangs leere Lehrfenster sind acht gegen sieben Takte vorhergesagt. Ein globaler Zähler umfasst weitere Befehle; konkrete Ripes-Zahlen werden ohne Lauf nicht behauptet.